Building an In-Memory Cache for CloudVault

A self-guided project handbook: problem, design, build plan and review process

Author: Abhinav

Project 2 of the Build-From-Scratch Systems Portfolio

Draft v1 - October 2026

Contents

How to Use This Handbook 3

Chapter 1: The Problem: Why Caches Exist 3

Chapter 2: What a Cache Is, and Is Not 4

Chapter 3: Where the Cache Fits in CloudVault 5

Chapter 4: Architecture: The Layers 6

Chapter 5: DSA Foundations You Need 7

Chapter 6: Expiry (Time-to-Live) 8

Chapter 7: Memory Limits and Eviction 8

Chapter 8: Keys and Invalidation 9

Chapter 9: Concurrency and Correctness in Node.js 10

Chapter 10: Optional: Persistence and a Standalone Server 11

Chapter 11: Testing Strategy 12

Chapter 12: Benchmarking and Measurement 13

Chapter 13: The Build Plan 14

Chapter 14: The Review Process 15

Chapter 15: Design Note Template 15

Chapter 16: Presenting the Project 16

Appendix A: Glossary 16

Appendix B: Self-Check Questions 17

Appendix C: Further Reading 17

How to Use This Handbook

This handbook guides you through designing and building an in-memory cache for CloudVault. It is written for self-directed learning: it explains the problem, the concepts and the design decisions, but it deliberately contains no finished code. You write the code, then have it reviewed against the checklists in this handbook.

The working method

1. Read a chapter and answer its questions in your own words, in a design note.
2. Write the code for that phase, with tests.
3. Run the acceptance checks listed for the phase.
4. Send the code, the design note and the test results for review.
5. Fix what the review finds, then move to the next phase.

What you will end up with

- A reusable cache module in JavaScript (Node.js) with per-key expiry, a memory limit, eviction, group invalidation and statistics.
- That module integrated into CloudVault where measurements show it helps.
- Benchmarks showing real before-and-after numbers.
- A design note and a project report section you can show to evaluators and employers.

> **Ground rules**
> Measure before you optimize. Keep each phase small and tested before starting the next. Prefer correctness over cleverness: a cache that serves stale or wrong data is worse than no cache.

Chapter 1: The Problem: Why Caches Exist

Most applications read the same data repeatedly: sessions, folder listings, counters, results of expensive calculations. Fetching all of it from a disk-based database on every request is slow, and it puts growing load on the database as traffic increases.

The speed gap

Source	Typical access time	Relative to RAM
RAM	about 100 nanoseconds	1x
SSD read	about 100 microseconds	roughly 1,000x slower
Network call to a database	about 1 to 10+ milliseconds	roughly 10,000x or more slower

These are rough orders of magnitude, not exact figures, and they vary by hardware. The point is the ratio: memory is thousands of times faster than going to disk or across a network. A cache keeps frequently used data in memory so most requests never reach the slower layer.

The cost side

A cache is not free. Every design decision in this handbook comes from one of these four costs:

- **Limited memory.** RAM is smaller than disk, so something must be removed when the cache fills up (eviction).
- **Stale data.** The cached copy can differ from the real data. Entries need expiry (TTL) and explicit invalidation.
- **Volatility.** RAM is lost on restart. A cache is usually allowed to lose data because the real data lives elsewhere.
- **Concurrency.** Many requests touch the cache at once, so correctness under overlapping operations matters.

Chapter 2: What a Cache Is, and Is Not

It is tempting to describe a cache as a database that keeps data in memory for a while. That is close, but the differences drive the design.

	Traditional database	Cache
Access	Queries over fields, tables, joins	Exact key lookup only
Structure	Schemas and relations	Keys and values
Storage	Disk, durable by default	RAM, lost on restart unless you add persistence
Role	Source of truth	A fast copy of data that lives elsewhere
Capacity	Limited by disk	Limited by memory, so it must evict
Expiry	Rare	Central feature, set per key

Per-key expiry

Expiry is not one global timer. Each key can have its own time-to-live, or none. A session may live 30 minutes, a disk-usage figure 10 seconds, a configuration value forever. Design expiry as a per-key setting.

The cache-aside pattern

In the most common setup the application, not the cache, talks to the database:

1. The application asks the cache for a key.
2. Hit: the cache returns the value immediately.
3. Miss: the application reads the database, then stores the result in the cache, optionally with a TTL.
4. The next request for the same key is a hit, until the entry expires or is evicted.

The cache itself only needs to store, return, expire and remove data. It does not need to know a database exists. A variant called read-through lets the cache fetch from the database on a miss; treat it as an extension, not the basic version.

The central question: staleness

What should happen if the data changes in the database while an older copy sits in the cache? There are three families of answers, each with trade-offs:

- **Wait for expiry.** Simple, but readers see old data until the TTL ends.
- **Invalidate on write.** When data changes, delete the affected cache entries. Fresher, but you must find every key the change affects.

- **Update on write.** Write the new value into the cache at the same time as the database. Fast reads, but the two writes can disagree if one fails.

Most real systems combine a sensible TTL (a safety net) with explicit invalidation (for freshness). Chapter 8 applies this to CloudVault.

Chapter 3: Where the Cache Fits in CloudVault

CloudVault is a single Node.js process on one machine, using SQLite in the same process. That shapes the design.

Decision: an in-process module, not a server

A cache that runs inside the same process avoids network latency and extra moving parts. A separate cache server over TCP would add complexity with no benefit for a single-process application. So the plan is:

1. Build the cache as a clean module with a small API.
2. Integrate it into CloudVault.
3. Optionally, later, wrap the same core in a TCP server as a separate learning project (Chapter 10).

Candidates to cache

Candidate	Why	Watch out for
Session validation	Checked on every request	A logged-out or revoked session must stop working immediately. A TTL alone is not enough; invalidate explicitly.
Folder listings	Repeated, costlier with many files	Invalidate on upload, delete, rename and move.
File metadata	Frequent lookups	Same invalidation rules as listings.
Disk usage stats	The filesystem call is repeated by the dashboard	A short TTL of a few seconds is enough.
Folder size totals	Expensive if computed by walking the tree	Invalidate up the parent chain when contents change.
Preview thumbnails	Costly to generate	Large, so they need a size cap and eviction.

> **Do not cache file contents**
> Files are large. Stream them from disk and let the operating system's own file cache do its job. Caching them in your process would waste memory and risk crashing a small device.

Constraints from the real environment

- **Limited memory.** On a Raspberry Pi or old laptop, memory is scarce. A maximum size and an eviction policy are required, not optional.
- **Multiple users.** Keys must be namespaced per user, and you need a way to delete groups of keys at once.
- **Security.** Staleness affects sessions and permissions, so a stale entry can be a security bug, not just an inconvenience.

Measure before you cache

SQLite in the same process is already very fast, often microseconds for simple queries. A cache in front of it may add little for simple lookups. It helps most for expensive operations: recursive folder sizes, large listings, thumbnail generation, filesystem statistics. The right order is:

1. Add simple timing logs around the slow paths.
2. Find what is actually slow.
3. Cache only that.
4. Compare before and after with a repeatable benchmark.

Even a finding such as "caching improved folder listing from X ms to Y ms but gave no gain on session checks" is a strong, honest result for your project report.

Chapter 4: Architecture: The Layers

Think of the cache as separate layers and build them separately. Mixing them is the most common beginner mistake.

Layer	Responsibility	Key questions
1. Storage engine	The in-memory data structure behind keys	How do you store entries with values, expiry times and usage information? What must be fast?
2. API layer	The functions the rest of CloudVault calls	What are the function names, inputs and return values? What does a miss return? Can callers tell "missing" from "stored but empty"?
3. Expiry	Removes outdated entries	How is each entry's deadline stored? How is the next expiry found quickly?
4. Memory control	Limits size and evicts	How is size measured? Which entry is removed first?
5. Observability	Statistics	How do you count hits, misses and evictions so you can prove it works?
6. Persistence (optional)	Survive restarts	Is it needed at all for a cache? What is the cost of losing it?
7. Network protocol (optional)	Standalone server	How are commands framed on a byte stream? Covered in Chapter 10.

For CloudVault you need layers 1 to 5. Layers 6 and 7 are extensions for the standalone-server version of the project.

Design the API on paper first

Before any code, write down every function the cache exposes: what it takes, what it returns, what it does on error, and whether it is synchronous. A good API is small. Typical needs are: store a value (with optional TTL), read a value, delete one key, delete many keys by group, check existence, clear everything, and read statistics. Decide these before you write the internals so you can change the internals later without breaking callers.

Chapter 5: DSA Foundations You Need

You do not need to relearn all of data structures and algorithms before starting. Learn the specific structures this project needs, and use the project to practice them.

Topic	Where it appears in the cache
Arrays, strings, buffers	Building keys, estimating entry sizes
Hash tables	The core key-to-entry store
Linked lists (singly and doubly)	LRU ordering, list-type values
Queues	Pending work, background tasks
Heaps / priority queues	Finding the next entry to expire
Binary trees, balanced trees	Sorted structures (stretch goal)
Time and space complexity	Reasoning about benchmarks

A three-week plan

Week	Learn	Build yourself
1	Big-O notation, arrays and strings, recursion basics	A hash table from scratch: hash function, collision handling, resizing. Do not use the built-in map.
2	Singly and doubly linked lists, stacks, queues	An LRU cache: a hash table plus a doubly linked list.
3	Binary trees and traversals, min-heap, binary search trees	An expiry scheduler built on a min-heap that always reports which item expires next.

> **How to study**
>
> Implement each structure yourself, write tests for it, then solve a handful of problems on it. Building first and puzzles second sticks far better than watching long courses. Start the cache's first version after week 1; do not wait for the plan to finish.

Later projects use more: Huffman coding needs a heap and a binary tree, version control needs graphs and hashing, and comparing file versions uses dynamic programming (longest common subsequence). Learn those when you reach them.

Chapter 6: Expiry (Time-to-Live)

Each entry can carry an expiry time. Handling expiry well needs two cooperating mechanisms.

Lazy expiry

When a key is read, check whether its deadline has passed. If so, treat it as missing and remove it. This is cheap and always correct for reads, but entries nobody reads again stay in memory forever.

Active expiry

A periodic background task removes expired entries even if nobody asks for them. Think about how often it runs and how much work it does per run, so it never freezes the application while sweeping a huge cache.

Question to answer: why is neither mechanism enough on its own? Write your answer in the design note.

Design questions

- Do you store a duration or an absolute deadline? What are the consequences for each?
- Which clock do you use? System time can jump (daylight changes, manual changes, clock sync). Look for a monotonic clock that only moves forward.
- How do you find the next entry to expire without scanning everything? This is where a min-heap helps.
- What happens when a key is overwritten with a new TTL, and the old deadline is still in your scheduler?
- Can a key expire at the exact moment it is being read or replaced? What result is correct?

Testing expiry without waiting

Do not make tests sleep. Make the clock a replaceable part of your cache (for example, passed in as a function) so a test can move time forward instantly. Designing for this from the start makes the cache far easier to test.

Chapter 7: Memory Limits and Eviction

A cache that grows without limit eventually crashes the process, which on a small device means taking CloudVault down. A limit and a removal policy are essential.

Deciding the limit

You can limit by number of entries, or by estimated memory. Counting entries is simple but ignores that one thumbnail can be larger than ten thousand small entries. JavaScript has no built-in size function, so you must estimate sizes yourself, for example from string length or buffer byte length. Decide your approach and note its inaccuracy.

Eviction policies

Policy	Idea	Good for	Cost
LRU (least recently used)	Remove what has not been used for longest	General use, simple to explain	Needs ordering updated on every read
LFU (least frequently used)	Remove what is used least often	Stable hot sets	Needs counters; old popular items can linger
FIFO	Remove the oldest inserted	Very simple	Ignores usage

Policy	Idea	Good for	Cost
TTL-first	Remove entries closest to expiry	Mixed TTLs	Needs expiry ordering

Implementing LRU

The classic design combines a hash table, for finding an entry in constant time, with a doubly linked list, for tracking usage order and moving an entry to the front in constant time. Work out on paper what happens on each operation: a read, a write of a new key, a write of an existing key, a delete, and an eviction. In JavaScript, the built-in Map remembers insertion order, which allows a simpler LRU. A good learning path is to build it with Map first, then rebuild it with your own list and compare.

Questions to answer

- Does reading an entry count as use? Does a failed lookup?
- When a new large entry arrives, how many small entries might you need to evict?
- What if one entry is larger than the entire limit?
- Do expired entries get evicted first, before any live entries?

Chapter 8: Keys and Invalidation

Good key design is what makes invalidation possible. If you cannot find every cache entry affected by a change, you will serve stale data.

Key naming

Use a consistent, structured scheme that includes the owner and the kind of data, so you can target groups. For example, a key can be built from parts such as kind, user and item identifier, joined with a separator you never allow inside the parts themselves. Decide your scheme and write it down. Two users must never be able to produce the same key.

Group invalidation

Some changes affect many entries: deleting a folder affects its listing, its parent's listing, size totals up the chain, and thumbnails of files inside it. You need a way to delete groups, by key prefix or by tags attached to entries. Compare both approaches: scanning all keys by prefix is simple but slow on large caches; tag indexes are faster but must be kept consistent.

Events and what they must invalidate

Fill in a table like this for your own design. The rows below are starting points to complete and challenge:

Event in CloudVault	Entries that may become stale
Upload a file	Listing of the target folder, folder size totals up the chain, disk usage stats
Delete or move to trash	Listings of source folder (and trash), size totals, metadata, thumbnails of the file, disk usage
Rename	Listing of the parent folder, metadata for the item
Move	Listings of both folders, size totals for both chains
Logout, password change, account removal	The user's sessions. This must be immediate.

Event in CloudVault	Entries that may become stale
Permission or sharing change	Anything cached per user that depends on access rights

> **Security rule**
> Never let a cache extend access beyond what the database allows. If a session is revoked, the cached copy must be removed at that moment, not when its TTL ends. A five-minute TTL on session validation without explicit invalidation would let a logged-out session keep working for up to five minutes.

Chapter 9: Concurrency and Correctness in Node.js

Node.js runs your JavaScript on a single thread. That makes simple synchronous cache operations atomic: nothing else can run in the middle of one function. But problems appear wherever your code waits (asynchronous operations), because other requests run during the wait.

Problem 1: the stampede

A popular entry expires. Fifty requests arrive before it is refilled. All fifty miss, and all fifty go to the database for the same data. Think about how to ensure only one request does the work while the others wait for its result. This is often called request coalescing or single-flight loading.

Problem 2: the stale write race

Request A misses the cache and starts reading old data. Meanwhile request B changes the data and invalidates the cache. Then A finishes and stores its old value in the cache. Now the cache holds stale data even though invalidation ran correctly. Think about how to detect this, for example by tracking a version or generation number for each key or group and refusing to store a value if the version changed.

> **A useful nuance**
> CloudVault's database library (better-sqlite3) is synchronous. If your read and your cache store happen in the same synchronous block with no waiting in between, Problem 2 cannot occur there. Check each use case: does any step in the middle wait? If not, the race does not apply, and you can say so in your design note.

Problem 3: check-then-act across waits

Code that checks the cache, awaits something, then writes based on the earlier check is acting on outdated information. Whenever you see an await between a check and a write, ask what could have changed.

Problem 4: unbounded work

A cleanup sweep that processes one million entries in one go blocks every other request. Break large jobs into small batches that yield back to the event loop.

Chapter 10: Optional: Persistence and a Standalone Server

These parts are not needed for CloudVault, but they complete the original goal of building a Redis-style system and teach networking and durability. Do them only after the in-process cache is finished and tested.

Persistence

The classic simple idea is an append-only log: record every change to a file, and replay the file at startup. Questions to work through:

- When do you force data to disk? Every write is safest but slow; periodic flushing is faster but can lose recent writes. What is acceptable for a cache?
- What if the machine crashes in the middle of writing a log entry? How do you detect and skip a half-written line?
- How do you stop the log growing forever? Look into rewriting the log from the current state (compaction).
- Alternative: periodic snapshots of the whole state. Compare restart speed, data loss and complexity.
- Should expired entries be written or replayed?

A standalone server

To turn the cache into its own server, you design your own protocol over TCP. The key concept: TCP is a stream of bytes, not a stream of messages. One send can arrive in pieces, and several sends can arrive glued together. Your server must find where one command ends and the next begins.

- **Line-delimited text:** easy to debug by hand, but values containing newlines need escaping.
- **Length-prefixed messages:** each message states its size first; handles any bytes, including binary, but is harder to read.

Also decide the concurrency model (event loop, thread per client, thread pool) and what happens with a client that sends half a command, sends an enormous value, or never disconnects.

Chapter 11: Testing Strategy

A cache that is fast but occasionally wrong is dangerous. Tests are the main defense.

Unit tests, per phase

- Store then read returns the same value; reading a missing key returns the documented miss result.
- Overwriting a key replaces the value and resets or keeps the TTL as documented.
- A key with a 2-second TTL is present at 1 second and absent at 3 seconds (use a fake clock).
- Deleting a missing key does not fail.
- Group invalidation removes exactly the intended keys and no others.
- At the size limit, adding an entry evicts the correct entry, and the evicted key is gone.
- A read refreshes recency for LRU; a failed read does not.
- Statistics counters match what the test did.

Edge cases to test deliberately

- Empty string keys and values; very long keys; keys containing your separator character.
- TTL of zero, negative, extremely large, or not a number.
- Values of different types: numbers, objects, null, undefined. Does the cache distinguish "stored null" from "missing"?
- Mutating an object after storing it: does the cached copy change too? Decide whether to copy on store and read.
- An entry larger than the whole memory limit.
- Thousands of operations in a row, then check the cache is still consistent.

Randomized testing

Compare your cache against a trivially simple reference model (a plain map with no limits) while performing thousands of random operations, and check that they agree wherever they should. Differences point to bugs in your LRU, expiry or invalidation.

Integration tests in CloudVault

- After upload, the folder listing shows the new file immediately.
- After deletion, the listing and the size totals update immediately.
- After logout, the old session cookie is rejected at once.
- Two different users never see each other's cached data.
- With the cache disabled, behavior is identical (only slower). This proves the cache is purely an optimization.

Chapter 12: Benchmarking and Measurement

Claims about speed must come from measurements. A benchmark is only convincing if it is repeatable and fair.

What to measure

Metric	Why it matters
Hit rate (hits divided by total lookups)	Shows whether the cache is actually used well
Latency, average and 99th percentile	The average hides slow outliers that users feel
Operations per second	Throughput of the cache itself
Memory used	Critical on small devices
Evictions and expirations	Show whether the limit and TTLs are sensible
End-to-end request time in CloudVault	The number that actually matters to users

Method

1. Define the workload: how many users, folders and files, what mix of reads and writes.
2. Warm up first (the first runs are slower), then measure many repetitions.
3. Run the same test with the cache off and on.
4. Change one thing at a time (limit size, TTL, policy) and record the effect.
5. Report the hardware you used. Results differ on a laptop and a Raspberry Pi.
6. Keep the benchmark script in the repository so others can repeat it.

Result table template

Operation	Without cache	With cache	Hit rate	Notes
Folder listing, 100 items				
Folder listing, 5,000 items				
Folder size total				
Session validation				
Disk usage stats				

Report negative results too. If an operation did not benefit, say so and explain why. That makes the positive results believable.

Chapter 13: The Build Plan

Eight phases. Each should work and be tested before the next begins. The acceptance checks are what you should be able to demonstrate.

Phase	What you build	Acceptance checks
0. Design	API on paper, key scheme, list of first things to cache, answers to the design note questions (Chapter 15)	A short written design note exists and has been reviewed.
1. Core store	Set, get, delete, existence check, clear	Unit tests pass, including missing keys and overwrites.
2. Expiry	Per-key TTL, lazy removal, periodic cleanup, replaceable clock	Time-based tests pass without real waiting; cleanup does not block.
3. Limits and eviction	Size limit, LRU eviction	Eviction order verified by tests; oversize entry handled.
4. Group invalidation	Delete by prefix or tag	Only intended keys removed; no cross-user leakage.
5. Statistics	Hits, misses, evictions, expirations, size estimate	Counters match scripted scenarios.
6. Integration	Use it in CloudVault, one use case at a time, starting with the slowest measured operation	Integration tests pass; behavior identical with cache disabled.
7. Benchmark	Repeatable before-and-after benchmark	Results table filled in on at least one real device.

Stretch goals

- Replace the Map-based internals with your own hash table, linked list and heap, then compare benchmarks.
- Single-flight loading to prevent stampedes.
- Version checks to prevent stale-write races.
- A second eviction policy (LFU) and a comparison.
- Persistence and the standalone TCP server (Chapter 10).
- A small statistics page in the CloudVault dashboard.

Chapter 14: The Review Process

When you finish a phase, send the code, your design note and your test results. The review checks the following.

- **Correctness:** Does the code do what the API says in every case, including errors and edge cases?
- **Edge cases:** Empty inputs, huge inputs, repeated operations, unexpected types.
- **Concurrency:** Are there waits between checks and writes? Can stale data be stored?
- **Complexity:** Are the core operations as fast as they should be as the cache grows?
- **Memory behavior:** Can anything grow without bound? Are removed entries fully released?
- **Security:** Can one user reach another's data? Do revoked sessions stay valid?
- **Design:** Is the API small and clear? Are layers separated?
- **Tests:** Do they actually try to break the code, or only confirm the happy path?

After the review, you will receive a list of test scenarios designed to break your implementation. Hints are available on request: they point you in a direction without writing the code. A finished solution is provided only if you ask for one.

Chapter 15: Design Note Template

Write answers to these in your own words before coding the related phase. A short, honest note beats a long, vague one.

A. Scope

1. Which three or four things in CloudVault will you cache first, and what is each key and value?
2. For each, what does the benchmark say is slow today?
3. What will you explicitly not cache, and why?

B. Staleness

1. For each cached item, which events make it stale, and which operation must delete it?
2. What is the TTL for each, and why that number?
3. What is the worst case if an invalidation is missed?

C. Limits

1. What is the memory limit, and how will you estimate entry sizes?
2. What happens when the cache is full?
3. What happens if one entry is larger than the limit?

D. Correctness

1. Where in the code does it wait between a check and a write?
2. How do you prevent several requests refilling the same key at once?
3. How do you prevent an old value being stored after invalidation?

E. API

1. List every function with inputs, outputs and error behavior.
2. How does a caller tell "missing" from "stored but empty"?
3. Does the cache copy values, or hand out the stored object?

Chapter 16: Presenting the Project

The same work serves your diploma report, your portfolio and your resume. Describe what you built and what you measured, not just the name.

For your report and portfolio

- The problem: what was slow in CloudVault, with measurements.
- The design: architecture diagram, key scheme, invalidation table, trade-offs.
- The results: the benchmark table, including where the cache did not help.
- Limitations and future work, such as moving to a networked cache if CloudVault ever runs on several machines.

Resume wording (when the work is finished and measured)

Example shape: "Designed and implemented an in-memory cache module with per-key TTL, LRU eviction and group invalidation for a self-hosted cloud storage app; reduced folder-listing latency by X% on a Raspberry Pi." Use only numbers you measured.

> **Honest claims**
> Say what is true: an in-process cache module designed from first principles and integrated into a real application. Do not call it a Redis replacement. Precise claims are respected; inflated ones are not.

Appendix A: Glossary

Term	Meaning
Cache hit / miss	The requested key was found / not found in the cache.
Cache-aside	The application checks the cache, and on a miss loads from the database and stores the result.
Read-through	The cache itself loads data from the database on a miss.
TTL (time-to-live)	How long an entry stays valid after being stored.
Lazy expiry	Expired entries are removed when they are next accessed.
Active expiry	A background task removes expired entries.
Eviction	Removing entries to free memory when a limit is reached.
LRU / LFU	Least recently used / least frequently used eviction policies.
Invalidation	Deliberately removing entries that are no longer correct.
Stale data	A cached value that no longer matches the source of truth.
Stampede	Many simultaneous misses for the same key, all hitting the database.
Hit rate	Hits divided by total lookups.

Term	Meaning
Percentile latency (p99)	The time under which 99 percent of requests completed.
Monotonic clock	A clock that only moves forward, unaffected by system time changes.
Atomic operation	An operation that cannot be observed half-finished.
Append-only log	A file where changes are only added at the end, and replayed to rebuild state.

Appendix B: Self-Check Questions

Use these to test your understanding before asking for a review. If you cannot answer one, revisit the related chapter.

1. Why is a cache allowed to lose data, when a database is not?
2. Why can a TTL alone be a security problem for sessions?
3. Why are lazy and active expiry both needed?
4. What are the four operations of an LRU cache, and what does each do to the hash table and the list?
5. What can go wrong when your code waits between checking the cache and writing to it?
6. Why might a cache in front of an in-process SQLite database give little benefit for simple queries?
7. Why should tests use a fake clock?
8. What is the difference between a hit rate of 95 percent with a slow cache and 60 percent with a fast one, and which would you rather have?
9. What would have to change if CloudVault ran on two servers instead of one?
10. What would you measure to decide whether the memory limit is set correctly?

Appendix C: Further Reading

- **Designing Data-Intensive Applications** by Martin Kleppmann: caching, consistency and storage ideas explained clearly.
- **Introduction to Algorithms** by Cormen, Leiserson, Rivest and Stein: reference for hash tables, heaps and trees.
- **Official Node.js documentation:** the event loop and timers, to understand where waiting happens.
- **Redis documentation:** read how existing systems describe expiry and eviction, after you have designed your own, to compare your decisions with theirs.

Read these after forming your own design, not before. Comparing your decisions with established ones is more instructive when you have already made the decisions yourself.